Inhalt der Checkliste

Inhalt der Checkliste **1**
Projektabgabe Truck Signs API **2**
1. Repository 2
Dockerfile 2
docker-compose.yml 2
entrypoint.sh 2
2. Dokumentation 2
3. Hinweise 3

Projektabgabe - Truck Signs API v2

Bitte erfülle alle Punkte auf dieser Liste, bevor du das Projekt einreichst. Solltest du weitere Extras eingebaut haben, erwähne das kurz, damit sich die Mentoren dies bei Bedarf anschauen können.

1. Repository

Vorhandene Dateien

- ☑ Es ist eine `Dockerfile` vorhanden und entsprechend der Kriterien unten korrigiert worden
- ☑ Es ist eine `docker-compose.yml` vorhanden und entsprechend der Kriterien unten korrigiert worden
- ☑ Es ist eine `entrypoint.sh` vorhanden und entsprechend der Kriterien unten korrigiert worden
- ☑ Eine Datei `README.md` ist vorhanden und entsprechend der Kriterien unten angepasst worden
- ☑ Es befinden sich keine weiteren Dateien im Repository, ohne dass diese explizit in der `README.md` benannt und beschrieben werden

Dockerfile

- ☑ Das Base-Image verwendet einen gültigen Python-Tag
- ☑ Der `pip install`-Befehl referenziert die `requirements.txt` korrekt
- ☑ `EXPOSE` gibt eine Port-Nummer an
- ☑ Das `entrypoint.sh` Script wird im Dockerfile als ausführbar markiert
- ☑ Der `ENTRYPOINT` referenziert das Script mit vollständigem Pfad

docker-compose.yml

- ☑ Das Postgres-Image verwendet einen gültigen Tag
- ☑ Es gibt keinen `build` Context mehr, stattdessen wird ein Image referenziert
- ☑ Sensitive Werte sind nicht hardcoded
- ☑ Beide Services haben eine geeignete Restart Policy
- ☑ Du hast einen Port des Containers exposed, sodass der Container vom Internet aus erreichbar ist
- ☑ Der `backend`-Service hat ein Port-Mapping
- ☑ Beide Services befinden sich im selben Docker-Netzwerk
- ☑ Volumes sind korrekt konfiguriert, sodass Datenbankdaten nach einem Neustart erhalten bleiben und keine dauerhafte Verbindung vom Host in den Container besteht

entrypoint.sh

- ☑ Der `while`-Loop zum Warten auf die Datenbank enthält `sleep`, sodass das Script nicht in einer Busy-Loop hängt
- ☑ Das Script führt `python manage.py migrate` aus, bevor die Anwendung startet
- ☑ Erweitere das Skript so, dass automatisiert ein Superuser erstellt wird. Falls dieser schon vorhanden ist, sollte das Skript diesen Schritt überspringen

README.md

- ☑ Die README sollte ein Inhaltsverzeichnis a.k.a. eine Table-of-Contents (ToC) enthalten
  - ☐ Die einzelnen Sektionen sind in der ToC verlinkt
- ☑ Eine Sektion mit einer Beschreibung des Repositories muss vorhanden sein. In dieser Beschreibung sollte genannt werden was die wesentlichen Inhalte sind, was der Zweck des Repositories ist
- ☑ Eine Sektion "Quickstart" sollte als Teil der README enthalten sein. Hier sollen kurz die Voraussetzungen genannt und eine Schnellstart-Anleitung beschrieben sein.
  - ☑ es sollte hierbei eine sektion how-to-build-the-image geben
- ☑ Es soll eine ausführliche Variante der vorgenannten Sektion als "Usage" enthalten sein. Hier soll genauer auf die Konfiguration und Konfigurierbarkeit eingegangen werden, d.h. es soll auch erklärt werden, wie relevante Passagen modifiziert werden können, um andere Resultate zu erzielen.
  - ☑ Es muss dokumentiert sein, wie ein Container Image erzeugt werden kann
  - ☑ Der **docker run** Befehl muss dokumentiert sein - env-variablen oder andere sensitive information sollte durch Platzhalter ersetzt werden

2. Dokumentation

Die Dokumentation des Codes, sowie des Projektes soll im Repository in Form einer README Datei stattfinden.
Die Dokumentationssprache für alle Projekte (und zugehörige Unterlagen) ist englisch.

3. Hinweise

Allgemeine Hinweise

- ☑ Zusätzlich zu deinem GitHub Repository solltest du ein kurzes Loom Video (maximal 5 Min) aufnehmen und bereitstellen, indem du kurz deine Abgabe zeigst und vorstellst was du getan hast - dabei musst du nicht alle Details erwähnen, jedoch sollst du auf alle relevanten Schritte kurz eingehen und diese zeigen.

Sicherheitshinweise

- ☑ Speichere keine SSH-Keys im Workspace deines Git-Repositories
- ☑ Speichere keine Passwörter, Tokens, oder Benutzernamen in deinem Code. Verwende hierfür stattdessen Environment-Variablen
- ☑ Speichere keine IP-Adressen, oder sonstigen sensiblen Informationen in einem Git Repository

Code-Konventionen

- ☑ Für build-args, environment Variablen und Shell-Variablen gilt folgende Namenskonvention: UPPER_CASE_WITH_UNDERSCORE
- ☑ Bei einer Referenz auf eine Variable sollte immer die {}-Notation verwendet werden um Fehler in der Interpretation zu vermeiden: ${SOME_VAR_VALUE}, statt: $SOME_VAR_VALUE
- ☑ Es sollten für Build-args, oder Environment Variablen "Default"-Werte konfiguriert werden, allerdings nur dann, wenn dies Sinn ergibt.
- ☑ Kritische Konfiguration wie Tokens, Passwörter oder ähnliches sollte nicht im Code-Repository gespeichert sein, sondern bspw. durch die Verwendung eines .env-files in einen Container hineingegeben werden

Testing

Bevor Du dein Projekt einreichst, solltest du die folgenden Dinge sicherstellen und getestet haben:

- ☑ Die Truck Signs API ist erreichbar unter der IP-Adresse deiner Cloud-VM auf Port 8020
- ☑ Dein ENTRYPOINT startet die WSGI Applikation
- ☑ Nach einem Neustart des Setups, sind die konfigurierten Daten noch vorhanden und werden nicht gelöscht oder überschrieben
- ☑ Die Container werden neu gestartet, sobald ein Fehler passiert, der zum Terminieren des Containers führt.
- ☑ Es gibt keinen doppelten Startbefehl für `gunicorn`